



Universidad de
Castilla~La Mancha



Plan de Mantenimiento

Ingeniería del Software II
NETA-SA

Integrantes:

***Mario Castro Hernández, Javier Calvo Díez, Álvaro Rayo Gallego y Alberto Corroto
Fernández***

Índice

1. Introducción	3
2. Mantenimiento.	4
2.1 Quality Rules	4
2.2 Quality Gates	6
2.3 Tipos de Mantenimiento	8
3. Aspectos de calidad tratados.	10
4. Análisis Final.	16

1. Introducción

Este plan de mantenimiento se ha llevado a cabo con el fin de aportar valor a la capacidad de nuestro proyecto mediante su modificación conservando la integridad del mismo. Para realizar este mantenimiento, se ha usado a forma de soporte la herramienta de SonarQube Server la cuál nos ha permitido medir distintos aspectos de calidad de nuestro proyecto para determinar en qué partes podemos mejorar respecto a lo que ya se había conseguido.

La introducción de este plan de mantenimiento se llevó a cabo durante la versión 1.0 de nuestra aplicación y a través de este plan de mantenimiento se detallarán los diferentes tipos de mantenimiento por lo qué se ha optado y cómo han sido aplicados al proyecto, tomando como apoyo diferentes imágenes para aportar credibilidad y verificar los hechos.

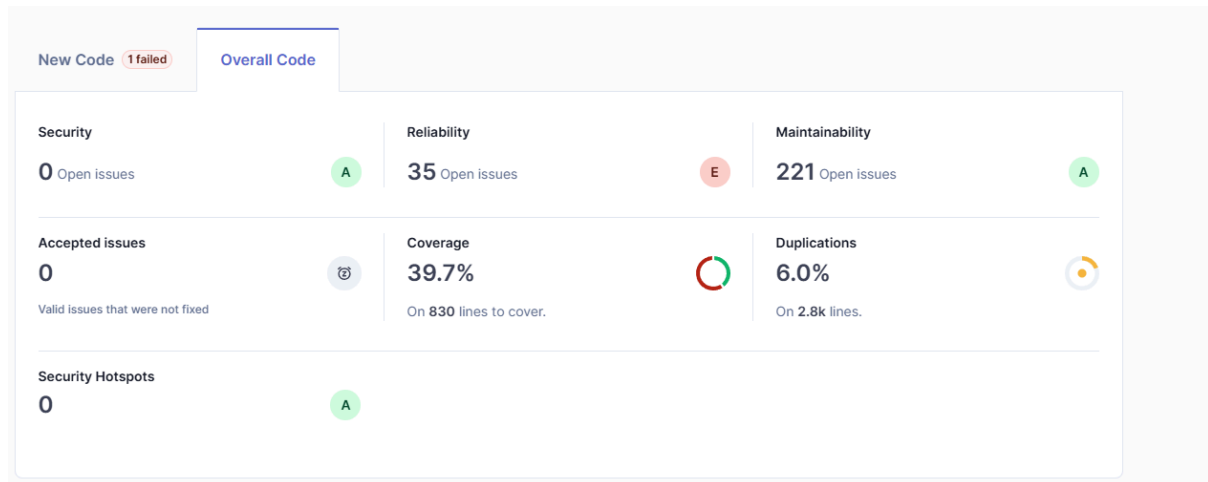
Antes de empezar, nos gustaría detallar que debido a diversos problemas con la versión gratuita de SonarQube Cloud los cuáles empezaron a entrar en vigor a finales de 2024, se ha decidido por montar y gestionar un servidor de SonarQube Community Server con la ayuda del sitio de hosting de VMs llamado DigitalOcean, el cuál nos ha permitido durante un periodo de prueba de 2 meses, crear una VM lo suficientemente competitiva para poder correr en ella un Servidor de SonarQube y realizar con él la CD/CI junto a Github.

Hay que tener en cuenta que no se verá directamente vinculado en el apartado de “Actions” del repositorio dentro de Github ya que no es algo que permita la edición Community, sin embargo, cómo se ha mencionado antes, sí que permite la CD/CI con Github por lo que nos es suficiente.

A forma de resumen, hemos integrado SonarQube con Github consiguiendo CD/CI y hemos empleado la plataforma DigitalOcean para alojar dicho servidor Sonar.



Para dar cierto contraste al final del documento frente al análisis final, cabe indicar que uno de los primeros análisis que se tuvo en Sonar tuvo como resultado:



En base a este análisis, discutiremos los aspectos de calidad y tipos de mantenimiento tratados más adelante de este documento para ver cómo hemos decidido mejorar esto y cómo nos ha quedado teniendo en cuenta dichas soluciones en el análisis final.

2. Mantenimiento.

2.1 Quality Rules

Las Quality Rules definen cómo el código debe de escribirse para poder cumplir con los diversos estándares de calidad, seguridad y buenas prácticas, estas reglas vienen definidas por defecto en base al lenguaje empleado por Sonar. Sin embargo, hemos decidido incluir ciertas reglas adicionales a las que trae SonarQube para el Quality Profile de Java.

Para esto anterior, hemos realizado una copia de las reglas dicho Quality Profile e incluido las que hemos encontrado que aportan mayor beneficio a nuestro proyecto. Dichas reglas se van a ejecutar cuando realicemos un análisis y nos ayudarán a reportar los bugs, vulnerabilidades y code smells encontrados. Dicho perfil personalizado lo hemos denominado “Neta Rules”.

Quality Profiles

Quality profiles are collections of rules to apply during an analysis. For each language there is a default profile. All projects not explicitly assigned to some other profile will be analyzed with the default. Ideally, all projects will use the same profile for a language. Learn more about [Quality Profiles](#) .

Filter by  

Java, 2 profile(s)	Projects 	Rules	Updated	Used	
NETA Rules	DEFAULT	539	15 days ago	10 days ago	
Sonar way BUILT-IN	0	522	18 days ago	15 days ago	

Cómo observamos, hemos incluido 17 reglas adicionales a las que nos aporta por defecto Sonar en su perfil para Java. Si comparamos ambos perfiles, podemos ver las reglas que hemos añadido, siendo estas:

Quality Profiles / Java / NETA Rules

NETA Rules DEFAULT

Compare with

NETA Rules has **17 additional rules** than Sonar way

17 rules in NETA Rules

"main" should not "throw" anything Intentionality Maintainability 	Deactivate
"switch case" clauses should not have too many lines of code Adaptability Maintainability 	Deactivate
Checked exceptions should not be thrown Consistency Maintainability 	Deactivate
Spring components should use constructor injection Consistency Maintainability 	Deactivate
Enum values should be compared with "==" Consistency Maintainability 	Deactivate
Classes should not have too many "static" imports Adaptability Maintainability 	Deactivate
Comments should not be located at the end of lines of code Consistency Maintainability 	Deactivate
Equality operators should not be used in "for" loop termination conditions Intentionality Maintainability 	Deactivate
"if ... else if" constructs should end with "else" clauses Intentionality Maintainability 	Deactivate
Methods should not have too many lines Adaptability Maintainability 	Deactivate

Methods should not have too many lines	Adaptability	Maintainability	Deactivate
Classes should not have too many methods	Adaptability	Maintainability	Deactivate
Control flow statements "if", "for", "while", "switch" and "try" should not be nested too deeply	Adaptability	Maintainability	Deactivate
Exit methods should not be called	Consistency	Maintainability	Deactivate
"==" and "!=" should not be used when "equals" is overridden	Intentionality	Maintainability	Deactivate
Constructor injection should be used instead of field injection	Intentionality	Reliability	Deactivate
"Exception" should not be caught when not required by called methods	Intentionality	Maintainability	Deactivate
Spring "@Controller" classes should not use "@Scope"	Intentionality	Reliability	Deactivate

Cómo observamos, la gran mayoría están relacionadas con la mantenibilidad (Maintainability) y algunas pocas con la confiabilidad (Reliability). Gran parte de las que se han elegido corresponden a una severidad media, siendo algunas pocas de severidad baja o incluso alta y bloqueante. Casi todas esas reglas corresponden a aspectos de Java pero algunas también han sido añadidas para tratar algunos aspectos de Spring.

2.2 Quality Gates

Las Quality Gates son un conjunto de condiciones de aceptación usadas para determinar si una versión de código puede considerarse apta para ser liberada o integrada. Estas condiciones pueden ser personalizadas en Sonar y para ello, **hemos creado una Quality Gate personalizada mediante la cual se evaluarán los resultados de nuestras reglas "Neta Rules" y si todo está correcto, nos indicará si es apropiado incorporar el nuevo código o si el código actual ha pasado todas las reglas y condiciones que hemos especificado.**

Teniendo en cuenta todo esto anterior, las condiciones que hemos considerado oportunas para nuestra Quality Gate llamada **"Sonar_NETA"** son las siguientes:

Sonar_NETA
DEFAULT

⚙️ This quality gate complies with Clean as You Code

This quality gate is [configured for Clean as You Code](#). It ensures that:

- ✓ New code has 0 issues
- ✓ All new security hotspots are reviewed
- ✓ New code has sufficient test coverage
- ✓ New code has limited duplications

Conditions ⓘ

Conditions on New Code

Metric	Operator	Value
Issues	is greater than	0
Security Hotspots Reviewed	is less than	100%
Coverage	is less than	80.0% 
Duplicated Lines (%)	is greater than	3.0% 

Conditions on Overall Code

Metric	Operator	Value
Coverage	is less than	50.0%  
Issues	is greater than	200  
Security Hotspots Reviewed	is less than	100%  

Con estas condiciones, aseguramos que no haya ningún issue, ningún problema de seguridad, que la cobertura sea superior a 80% y que haya menos de un 3% de líneas duplicadas. Esto anteriormente dicho lo comprobamos para el nuevo código que introducimos, pero **para el código en su totalidad**, hemos decidido asegurarnos que la **cobertura sea mayor de 50%, que no haya ningún issue y que no haya ningún problema de seguridad.**

Además, podemos decidir en qué momentos queremos que nuestro código tanto nuevo como actual pase por nuestra Quality Gate. **Es decir, podemos decidir cuándo queremos realizar un análisis de Sonar. Esto lo hemos definido en el siguiente archivo en el cual especificamos un workflow a seguir por Github (cómo si lo tuviésemos vinculado en Github Actions) y es lo que permite la CI/CD con nuestro servidor de Sonar:**

```

Code Blame 36 lines (29 loc) · 880 Bytes

1  name: SonarQube Analysis
2
3  on:
4    push:
5      branches: [ master-1.0, development ]
6    pull_request:
7      branches: [ master-1.0, development ]
8    workflow_dispatch:
9
10 jobs:
11   sonar:
12     runs-on: ubuntu-latest
13
14     steps:
15       - uses: actions/checkout@v3
16
17       - name: Set up JDK 17
18         uses: actions/setup-java@v3
19         with:
20           java-version: '17'
21           distribution: 'temurin'
22
23       - name: Cache SonarCloud packages
24         uses: actions/cache@v3
25         with:
26           path: ~/.sonar/cache
27           key: ${{ runner.os }}-sonar
28           restore-keys: ${{ runner.os }}-sonar
29
30       - name: Build with Maven
31         run: mvn clean install
32         working-directory: library
33
34       - name: Run SonarQube analysis
35         run: mvn sonar:sonar -Dsonar.login=${{ secrets.SONAR_TOKEN }} -Dsonar.host.url=${{ secrets.SONAR_HOST_URL }}
36         working-directory: library

```

Con esto, se realizará un análisis sonar cuando se produzca tanto un push como una pull request en las ramas master y develop, además, también nos va permitir poder elegir la rama que queramos analizar a nuestra elección dentro del Github Actions.

2.3 Tipos de Mantenimiento

Respecto a la naturaleza del mantenimiento empleado y su objetivo, con el uso de la herramienta de Sonar se han empleado mantenimiento correctivo y perfectivo. Cada uno se ha enfocado de la siguiente forma:

-Mantenimiento Correctivo:

Está relacionado con la corrección de errores o bugs que detectamos en el código y que potencialmente pueden llevar a fallos, estos fallos pueden tener diversos tipos de causas y SonarQube nos ayuda a detectarlas para poder corregirlos y mejorar los diversos aspectos de calidad que trataremos en un apartado posterior de este plan de mantenimiento.

Esto anteriormente mencionado, lo podemos ver reflejado tras un análisis en SonarQube denominados dichos bugs o fallos cómo “issues”.

The screenshot displays the SonarQube interface. On the left, a sidebar shows filters for 'My Issues' and 'All'. Below this, a 'Filters' section lists 'Issues in new code'. The 'Software Quality' section shows counts for Security (0), Reliability (35), and Maintainability (221). The 'Severity' section shows counts for Blocker (1), High (82), Medium (74), Low (65), and Info (0). The 'Clean Code Attribute' section shows counts for Consistency (143), Intentionality (62), Adaptability (17), and Responsibility (0).

The main area shows a list of issues. The first issue is 'Remove this commented out code.' with a severity of 'Maintainability' and a status of 'Open'. The second issue is 'Remove this duplicated import.' with a severity of 'Maintainability' and a status of 'Open'. The third issue is 'Move this trailing comment on the previous empty line.' with a severity of 'Maintainability' and a status of 'Open'. The fourth issue is 'Remove this unused import 'es.uclm.library.business.entity.Cliente'' with a severity of 'Intentionality' and a status of 'Open'.

Cómo observamos, Sonar los clasifica en base a su severidad y los aspectos de calidad que trata como la consistencia, adaptabilidad, intencionalidad, fiabilidad, mantenibilidad, etc....

Por lo tanto, para este tipo de mantenimiento se ha decidido llevarlo a cabo centrándonos en los issues que Sonar detecta e irlos resolviendo en base a su severidad. Como el mantenimiento es correctivo, nos centraremos en aquellos issues que causen errores o fallos.

-Mantenimiento Perfectivo:

Este mantenimiento es tratado por la mejora continua que empleamos junto a Github. Tiene el mismo enfoque que el mantenimiento correctivo explicado anteriormente en cuanto a enfocarnos en solucionar los “issues” que nos indica Sonar, **sin embargo, nos centraremos en los issues que al solucionarlos aporten beneficios para los aspectos de calidad que tratamos en este plan de mantenimiento como la mantenibilidad y fiabilidad. Esto quiere decir que este mantenimiento se centra en los issues que a la larga no lleven a fallos, pero si afectan aspectos de calidad.**

Este será en gran parte el principal tipo de mantenimiento llevado a cabo, ya que casi todos los issues están relacionados con fiabilidad y mantenibilidad. Por esto mismo, para llevarlo a

cabo eliminaremos code smells, refactorizaremos código duplicado y mejoraremos la cobertura de las pruebas entre otras muchas soluciones.

Además, usando Sonar, podemos enfocarnos en los issues que deben ser tratados por este tipo de mantenimiento **si filtramos por fiabilidad y mantenibilidad** y por ejemplo, nos podemos centrar en los que tengan **severidad alta o bloqueante**.

The screenshot displays the SonarQube web interface. On the left, the 'Filters' sidebar is active, showing 'Software Quality' with 'Maintainability' at 82 and 'Severity' with 'High' at 82. The main panel shows a list of issues for the file 'src/.../library/business/controller/GestorLogin.java'. The first issue is 'Refactor this code to not nest more than 3 if/for/while/switch/try statements.' with a 'Maintainability' quality gate and a 'brain-overload' tag. The second issue is 'Add a call to "setComplete()" on the SessionStatus object in a "@RequestMapping" method.' with a 'Reliability' quality gate and a 'spring' tag. The third issue is 'Define a constant instead of duplicating this literal "email" 7 times.' with a 'Maintainability' quality gate and a 'design' tag. The fourth issue is 'Define a constant instead of duplicating this literal "total" 3 times.' with a 'Maintainability' quality gate and a 'design' tag. The fifth issue is 'Define a constant instead of duplicating this literal "message" 7 times.' with a 'Maintainability' quality gate and a 'design' tag. Each issue has a 'Bulk Change' button, a 'Select Issues' dropdown, and a 'Navigate to issue' button. The issues are listed with their IDs (L55, L26, L108, L169, L203), effort (10min, 15min, 16min, 8min, 16min), and age (18 days ago).

3.Aspectos de calidad tratados.

A lo largo de este documento, se han mencionado diversos aspectos de calidad que se han tenido en cuenta para mejorar a través del mantenimiento. En este apartado, aportamos una definición breve de dichos aspectos y proporcionaremos soluciones que pueden mejorar dichos aspectos. Dichas soluciones además, nos las aporta ya Sonar y nos indica a qué se deben y qué podemos realizar para aplicarlas.

-Confiabilidad (Reliability):

La confiabilidad hace referencia a la capacidad que tiene el software de funcionar correctamente bajo determinadas condiciones en un tiempo específico. Gracias a este

aspecto, podemos minimizar fallos operativos, mejorar tolerancia a fallos y mejorar la disponibilidad

En lo referente a issues que conciernen a este aspecto, nos centraremos principalmente en los 2 únicos tipos que ha encontrado nuestro Sonar. Todos menos 1 issue relacionados con la confiabilidad que se han encontrado son de severidad media, y están relacionados con la sustitución del uso de “@Autowired” por un constructor explícito:

Remove this field injection and use constructor injection instead. [🔗](#)

Field dependency injection should be avoided [java:S6813](#) [🔗](#)

Line affected: L31 • Effort: 5min • Introduced: 18 days ago

☐ Open ▾
 ☐ Not assigned ▾
 ☐ No tags +

Where is the issue?
 Why is this an issue?
 How can I fix it?
Activity
More info

Software qualities impacted

Reliability ⚠️ Maintainability ⚠️

Clean code attribute

Consistency | Not conventional

Use constructor injection instead.

By using constructor injection, the dependencies are explicit and must be passed during an object's construction. This avoids the possibility of instantiating an object in an invalid state and makes types more testable. Fields can be declared final, which makes the code easier to understand, as dependencies don't change after instantiation.

Noncompliant code example

```

public class SomeService {
    @Autowired
    private SomeDependency someDependency; // Noncompliant

    private String name = someDependency.getName(); // Will throw a NullPointerException
}

```

Compliant solution

```

public class SomeService {
    private final SomeDependency someDependency;
    private final String name;

    @Autowired
    public SomeService(SomeDependency someDependency) {
        this.someDependency = someDependency;
        name = someDependency.getName();
    }
}

```

La solución sería definir un constructor explícito para cada “@Autowired” que hay, sin embargo, cómo esto es una ventaja que se nos ha introducido al usar Spring y simplifica el código evitando complejidad aunque pudiendo dificultad a futuro el testing, se ha omitido.

Con esto, se nos queda 1 issue de severidad bloqueante relacionado con la confiabilidad el cual sí vamos a abordar y tratar:

Add a call to "setComplete()" on the SessionStatus object in a "@RequestMapping" method. [🔗](#)

"@Controller" classes that use "@SessionAttributes" must call "setComplete" on their "SessionStatus" objects [java:S3753](#)



Line affected: L26 • Effort: 15min • Introduced: 18 days ago

Software qualities impacted

Reliability

Clean code attribute

Intentionality | Not complete

☐ Open ☐ Not assigned ☐ spring

Where is the issue? Why is this an issue? Activity

object must be passed ter.

library > src/.../es/uclm/library/business/controller/GestorPedidos.java

Open in IDE

[See all issues in this file](#)



```

21 import java.util.UUID;
22 import java.util.stream.Collectors;
23
24 @Controller
25 @RequestMapping("/RealizarPedido")
26 @SessionAttributes("pedidoItems")
27
28 public class GestorPedidos {
29
30     private static final Logger logger = LoggerFactory.getLogger(GestorPedidos.class);
31
32     @Autowired
33     private PedidoService pedidoService;
34
35     @Autowired
36     RepartoService RepartoService;

```

Add a call to "setComplete()" on the SessionStatus object in a "@RequestMapping" method.

Este issue tiene que ver con el almacenamiento de la información relacionada los ítems del pedido entre sesiones y indica que los datos almacenados entre sesiones deberían ser limpiados una vez acabe la sesión llamando a "setComplete()" ya que ni Spring ni JVM saben el momento oportuno para hacer esto. En nuestro caso, **el momento oportuno donde debería hacerlo es cuando el cliente ha pagado ya el pedido, es decir, ya no hace falta que se mantenga información entre sesiones de los "items" que pidió.**

Esto lo hemos incluido antes de redirigir al cliente a otra página al terminar el pago. Esto lo vemos aquí:

```

RepartoService.actualizarRepartidor(repartidor);

logger.info("Pago realizado con éxito para el pedido: " + pedidoId);
model.addAttribute( attribute: "message", attributeValue: "Pago realizado con éxito");
sessionStatus.setComplete(); //Indica a JVM que borre los datos de la sesion
return "redirect:/";
}

```

-Seguridad (Vulnerabilities):

Este aspecto evalúa la capacidad del software de proteger la información y evitar accesos no autorizados. Sin embargo, debido a que en los análisis que hemos realizado no hemos encontrado algún issue relacionado con este tipo de aspecto, no lo comentaremos mucho.

Sin embargo, sí que hay un issue en Security detectado por Github y es en referente a la versión de Derby usada:

The screenshot shows a Dependabot alert titled "Apache Derby: LDAP injection vulnerability in authenticator #1". The alert is marked as "Open" and was opened 4 months ago. It points to a dependency in a Maven library/pom.xml file.

Upgrade org.apache.derby:derby to fix 1 Dependabot alert in library/pom.xml
Upgrade org.apache.derby:derby to version 10.16.1.2 or later. For example:

```
<dependency>
  <groupId>org.apache.derby</groupId>
  <artifactId>derby</artifactId>
  <version>10.16.1.2</version>
</dependency>
```

Create Dependabot security update

Package	Affected versions	Patched version
org.apache.derby:derby (Maven)	>= 10.16.0.0, < 10.16.1.2	10.16.1.2

A cleverly devised username might bypass LDAP authentication checks. In LDAP-authenticated Derby installations, this could let an attacker fill up the disk by creating junk Derby databases. In LDAP-authenticated Derby installations, this could also allow the attacker to execute malware which was visible to and executable by the account which booted the Derby server. In LDAP-protected databases which weren't also protected by SQL GRANT/REVOKE authorization, this vulnerability could also let an attacker view and corrupt sensitive data and run sensitive database functions and procedures.

Mitigation:
Users should upgrade to Java 21 and Derby 10.17.1.0.

Alternatively, users who wish to remain on older Java versions should build their own Derby distribution from one of the release families to which the fix was backported: 10.16, 10.15, and 10.14. Those are the releases which correspond, respectively, with Java LTS versions 17, 11, and 8.

Severity: Critical 9.8 / 10

CVSS v3 base metrics

Metric	Value
Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	High
Integrity	High
Availability	High

EPSS score: 0.045% (13th percentile)

Tags: Runtime dependency, Patch available

Weaknesses: CWE-74, CWE-94

CVE ID: CVE-2022-46337

GHSA ID: GHSA-rqjc-c4pj-xorp

Aún así, cómo es la versión de Derby usada para las prácticas y empleada para la funcionalidad se ha decidido no cambiar nada ya que tampoco el proyecto va a estar expuesto al público y no se producirá ningún ataque malicioso. Sin embargo, si fuera un proyecto real, habría que poner el foco en solucionar este aspecto de seguridad.

-Mantenibilidad (Maintainability):

La mantenibilidad permite medir la facilidad con la que el software es entendido, corregido, mejorado o adaptado y es clave en el mantenimiento. Además, esto lo vemos ya que la gran parte de issues que indica Sonar tienen que ver con la mantenibilidad y estos no suponen un impacto en las funcionalidades cómo tal, pero sí en la deuda técnica que se va generando.

Para simplificar, los principales issues que encontramos y de mayor severidad tienen que ver con sustituir palabras que se repiten un gran número de veces por una variable o refactorizar ciertas expresiones. Esto lo vemos en Sonar:

The screenshot displays the SonarQube web interface. On the left, the 'Filters' sidebar is active, showing 'Software Quality' with 'Maintainability' at 82, and 'Severity' with 'High' at 82. The main panel shows a list of issues under the 'Maintainability' category. The issues are related to code duplication and are categorized as 'High' severity. The issues are:

- Refactor this code to not nest more than 3 if/for/while/switch/try statements.** (L34 - 10min effort - 18 days ago)
- Define a constant instead of duplicating this literal "errorMessage" 3 times.** (L73 - 10min effort - 18 days ago)
- Define a constant instead of duplicating this literal "email" 5 times.** (L37 - 12min effort - 18 days ago)
- Define a constant instead of duplicating this literal "error" 11 times.** (L40 - 24min effort - 18 days ago)
- Define a constant instead of duplicating this literal "cartasMenu" 8 times.** (L53 - 18min effort - 18 days ago)

The issues are all marked as 'High' severity and are related to 'Maintainability'. The interface also shows a 'Clean Code Attribute' section with 'Consistency' at 65, 'Intentionality' at 1, 'Adaptability' at 16, and 'Responsibility' at 0.

Los issues relacionados con la repetición se puede intuir fácilmente por qué son issues, por lo que nos enfocaremos por ejemplo en el de refactorizar que vemos en la anterior captura:

Refactor this code to not nest more than 3 if/for/while/switch/try statements.

Control flow statements "if", "for", "while", "switch" and "try" should not be nested too deeply [java:S134](#)

Line affected: L73 • Effort: 10min • Introduced: 18 days ago

☐ Open
 ☐ Not assigned
 ☐ brain-overload

Where is the issue?
 Why is this an issue?
 How can I fix it?
 Activity
 More info

```

53      entityManager.clear(); // Clear the EntityManager to avoid any stale data
54
55      Usuario usuario = loginService.findUsuarioById(email);
56      if (usuario == null) {
57          usuario = new Usuario();
58          usuario.setIdUsuario(email);
59          usuario.setPass(password);
60          usuario.setRol(role);
61          loginService.registerUsuario(usuario);
62      }
63
64      switch (role) {
65          case "cliente":
66              Cliente cliente = new Cliente(usuario, clienteNombre, apellidos, dni);
67              loginService.registerCliente(cliente);
68              break;
69          case "restaurante":
70              CodigoPostal codigoPostal;
71              try {
72                  codigoPostal = CodigoPostal.fromCode(codigoPostalStr);
73                  if (!codigoPostal.getLocation().equalsIgnoreCase(municipio)) {

```

Refactor this code to not nest more than 3 if/for/while/switch/try statements.

Software qualities impacted

Maintainability

Clean code attribute

Adaptability | Not focused

Básicamente, no es un error que afecte a la funcionalidad pero sí a la legibilidad del código y su mantenibilidad. Para solucionarlo, basta con refactorizar el código alineando mejor el try y el if para que estén al mismo nivel de profundidad y se vea.

-Testabilidad (Coverage).

En cuanto a la cobertura, hemos usado Junit junto a Surefire y la hemos controlado desde Sonar. **Para aumentar dicha cobertura, hemos desarrollado diferentes casos de prueba para después pasarlos en los test que hemos realizado principalmente a los controladores.** Nos hemos centrado en los controladores ya que son la parte fundamental de nuestra aplicación y hemos obviado otras partes que solo requerían test unitarios sin necesidad de desarrollar casos de prueba para ellos.

Hemos empleado una **cobertura del tipo each-use** para realizar dichos test, así mismo, todo lo relacionado con los casos de prueba se ha hecho en hojas excel adjuntas en la carpeta de documentación del proyecto, dichas hojas tienen esta forma:

Parámetro	Clases de Equivalencias	Valores de Clases de equivalencia
email	Presente / null	"cliente1@mail.com" / null
cliente	Cliente / null	Cliente / null
favoritos	Lista vacía / Lista con favoritos	[] / [Rest1, Rest2]
restaurantes	Lista vacía / Lista con varios	[] / [Rest1, Rest2, Rest3]

TABLA DE VALORES

Caso	email	cliente	favoritos	restaurantes
CU1	"cliente1@mail.com"	Cliente	[Rest1, Rest2]	[Rest1, Rest2, Rest3]
CU2	null	null	[]	[]

TABLA DE COBERTURA DE COMBINACION DE VALORES (Cada Uso)

library

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
es.uclm.library.business.controller		57 %		56 %	55 99	162 386	31 57	3 8
es.uclm.library.business.entity		68 %		25 %	93 170	155 378	89 166	1 16
es.uclm.library.business.service		3 %		0 %	42 44	62 64	36 38	3 5
es.uclm.library		0 %		n/a	2 2	3 3	2 2	1 1
Total	1.380 of 3.212	57 %	54 of 102	47 %	192 315	382 831	158 263	8 30

4.Análisis Final.

main

2.1k Lines of Code • Version 1.0.0 • [Set as homepage](#) [Take the Tour](#)

Quality Gate

✓

Passed

⚠

The last analysis has warnings. [See details](#)

Last analysis 3 minutes ago

New Code

Overall Code

Security

0 Open issues

A

Reliability

34 Open issues

C

Maintainability

126 Open issues

A

Accepted issues

0

🔍

Valid issues that were not fixed

Coverage

53.3%

🔄

Required ≥ 50.0%

On 831 lines to cover.

Duplications

6.0%

🔄

On 2.7k lines.

Security Hotspots

0

A